

2026/07/05

snow00two

JavaScript と p5.js¹

私のポートフォリオに私が JavaScript と p5.js で作ったアプリが置いてあります。その幾つかについて説明します。

1. JAVASCRIPT と P5.JS

1.1. **p5.js**. p5.js は Processing という Java 用の描画プログラミング言語を Javascript に移植したものである。以下のウェブ上のエディターで実行可能である。

<https://editor.p5js.org> (以下この色はハイパーリンク)

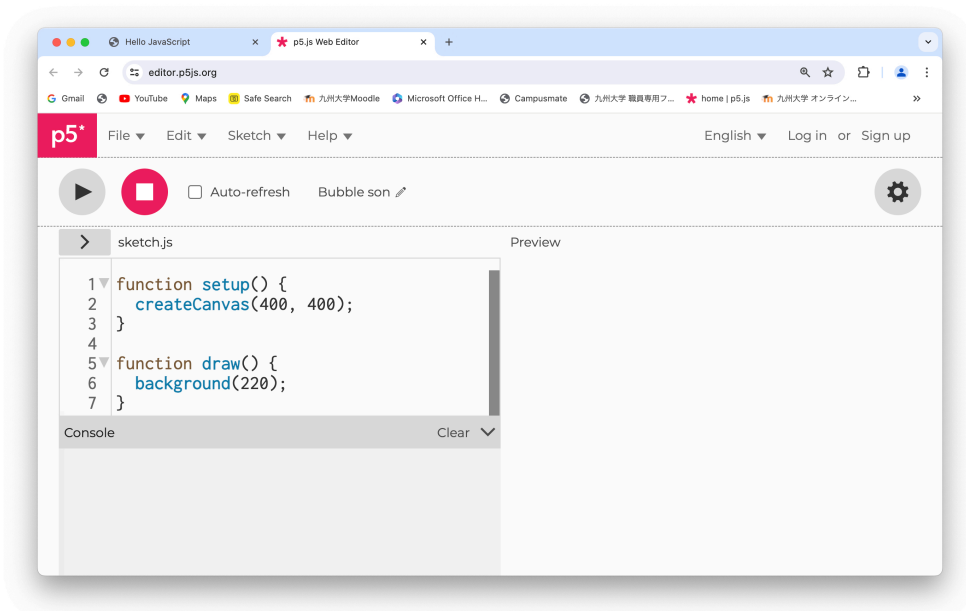


図 1. ウェブ上の p5.js エディター

公開した sketch.js を sketch.js の部分にコピーし、(ファイル octahedron.stl をアップロードし) 左上の三角のボタンを押すと右画面に描画が始まる。もちろん隣りの四角のボタンで止まる。ただし .js は JavaScript のファイルであることを示す拡張子 (extension) である。

また以下のようにすると自分のパソコン上で動かせる。p5.js というライブラリーをダウンロードすると p5 というディレクトリが作られる。公開された sketch.js をそのディレクトリーの sketch.js と置き換え、フリーのエディター **Visual Studio Code** でそのディレクトリ p5 を開き、Live Server の拡張を Visual Studio Code にインストールして Go Live のボタンを押すと、chrome で再生を始める。

¹GitHub version

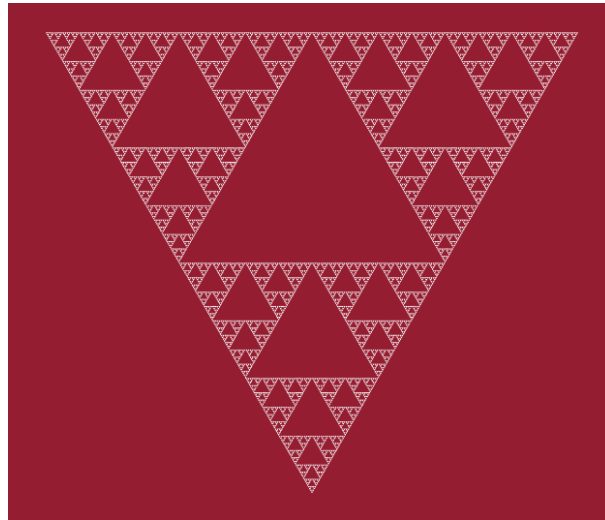


図 2. Sierpinski gasket

2. P5.JS の 2D アプリ

2.1. ソース. 以下が Sierpinski gasket を描くアプリのソースファイル sketch.js の内容である.

```
/* Sierpinski gasket etc., abbreviated version
 * 2026/06/22, by snow00two
 * \href{https://creativecommons.org/licenses/by-nc-nd/4.0/}{\ccbyncsa}
 */
/* canvas setting */
const WIDTH_CANVAS = 720 * 3/2 ; //=1080
const HEIGHT_CANVAS = 405 * 3/2 ; //=607.5
const CENTER_X = WIDTH_CANVAS/2 ;
const CENTER_Y = HEIGHT_CANVAS/2 ;

/* color setting */
const DARK_COLOR = [150,30,50];
const WHITE_COLOR = 255;
let colorChoice ;
let choiceC ;/* variable for color scheme */
let backColor ;
let frontColor ;

/* Sierpinski gasket setting, abbr. 3 */
const BASE_SEGMENT_3 = 500;
let height3;
let radius3;
let shift3Y;
let fixed3X;
```

```

let fixed3Y;

function setup() {
  /* Sierpinski gasket additional setting */
  height3 = sqrt(3)* BASE_SEGMENT_3 / 2; /* sqrt(3) is defined here */
  radius3 = 2 * height3 / 3 ; /* the radius of circumscribed circle */
  fixed3X = ( WIDTH_CANVAS - BASE_SEGMENT_3 ) / 2 ;
  fixed3Y = ( HEIGHT_CANVAS - height3 ) / 2 ;
  shift3Y= ( HEIGHT_CANVAS - height3 ) / 3 ;

  colorChoice = createSelect(); /* object setting color scheme */
  colorChoice.option('normal color','normal');
  colorChoice.option('reverse color','reverse');
  colorChoice.selected('normal');
  colorChoice.position(30, 8);
  colorChoice.changed( resetBackground ) ;

  frameRate(0.6);

  /* create canvas */
  createCanvas(WIDTH_CANVAS, HEIGHT_CANVAS);
  background(DARK_COLOR);
}

let n = 0; //clock parameter
/* main function controlling this application */
function draw() {
  let choiceC = colorChoice.value();
  if (choiceC == 'normal' ){
    backColor = DARK_COLOR;
    frontColor = WHITE_COLOR;
  } else {
    backColor = WHITE_COLOR;
    frontColor = DARK_COLOR;
  }
  background(backColor);

  translate(CENTER_X, CENTER_Y - shift3Y);
  fill(frontColor);
  iterateTriangleS(n);

  let I = 9 ; //Number of iterations
  if( n < I ){
    n++ ;
  } else {

```

4

```
        background(backColor);
        n = 0 ;
    }
}

/* recursive function creating Sierpinski gasket */
function iterateTriangleS(num){
    push();
    if (num == 0){
        fill(frontColor);
        noStroke();
        triangle(-BASE_SEGMENT_3 / 2, -radius3 /2,
            BASE_SEGMENT_3 /2, -radius3 /2, 0, radius3 );
        //Here we erase the center reverse triangle
    } else {
        scale( 1 / 2 );
        fill(frontColor);
        noStroke();
        for (let l=0;l < 3;l++){
            translate(0, radius3 );
            iterateTriangleS(num - 1);
            translate(0, -radius3 );
            rotate(2*PI/3);
        }
    }
    pop();
}

function resetBackground () {
    background(backColor);
    n = 0;
}
```

2.2. **一般のJavaScriptの動かし方.** JavaScript の処理の仕方は幾つかある. JavaScript はブラウザーに組み込まれているので, まず

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Hello JavaScript</title>
  </head>
  <body>
    <script type="text/javascript" src="test.js"></script>
  </body>
```

```
</html>
```

のような html のファイルを hoge.html として保存し、次に

```
window.alert('hello world!');
```

という行のみからなるファイルを test.js として hoge.html と同じディレクトリに保存する。ブラウザ Chrome のメニューバーの File のメニュー Open Files... から hoge.html を開くと

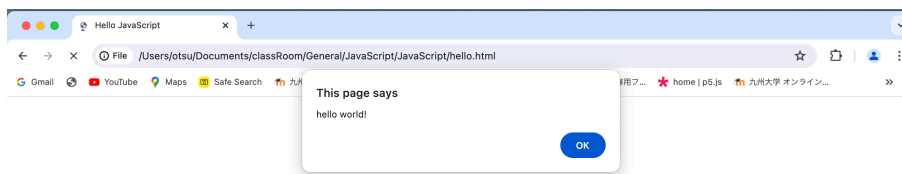


図 3.

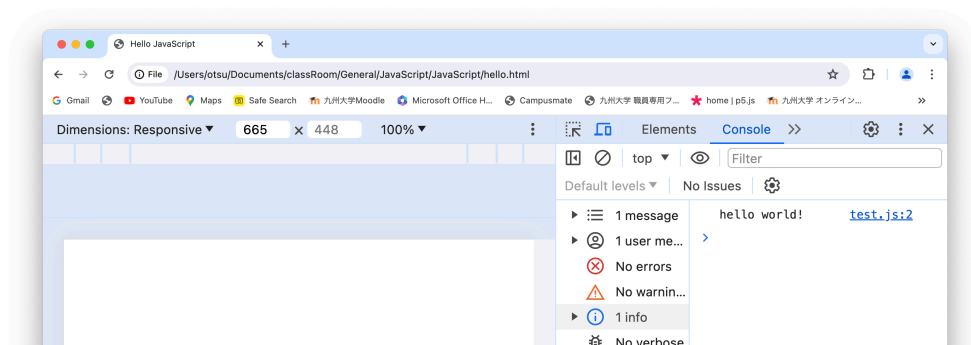


図 4.

図 3 のような結果が得られる。あるいは

```
console.log('hello world!');
```

を test.js として保存し、また hoge.html を開くと File のメニュー View から Developer の Developer Tools を確認すると図 4 のような結果が得られる。

他にも Terminal で test.js のあるディレクトリに移動し

```
% osascript test.js
```

で Terminal に結果が表示される。ただし、これは Mac のみで、逆に言うと Mac のスクリプト言語として JavaScript を使うことが出来る。一般的には Node.js をダウンロードしインストールすると

```
% node test.js
```

で Terminal に結果が表示される。したがって Mac を直接コントロールする必要が無いときは osascript より node.js を使うことをお奨めしたい。

2.3. JavaScript の言語仕様.

- JavaScript には変数の型はなく、文の終りで ; が必要
- let は変数の宣言、以前は var も使われていた.
- const は定数の宣言
- // はコメント文でこれ以降の文は無視される.
/* */ はブロックコメント文で途中の文は無視される.

よく使う制御構文：for 文の例：

```
let I = 5;
for (let i=0; i < I; i++) {
    console.log(i);
}
```

if 文の例：

```
let m = 80;
if ( m<60 ){
    console.log('F');
} else if ( m<70 ) {
    console.log('C');
} else if ( m<80 ) {
    console.log('B');
} else if ( m<90 ) {
    console.log('A');
} else {
    console.log('S');
}
```

function hoge(var1,...,var3) は関数 hoge の宣言で、続く {} の中に関数の定義を書く． var1,...,var3 が関数の変数で定義の中で使える．関数の中で定義された変数はローカル変数で、関数外で使われる変数はユニバーサルな変数になる．

2.4. p5.js の 2D モードの説明とソースファイルの解説. p5.js の 2 次元モードでは y 軸の方向が通常とは逆で下向きになっている．これはテレビなどの表示では、一次元で送られてくるデータを図 5 のように上から下に折り返しながら表示していたからである．したがって、円周上で角度が増加するプログラムでは時計回りに回転する．もちろん、反時計回りにするためにはパラメータの符号を代えれば良い．

関数 setup() の中でウィンドウ等を初期化している．createCanvas(widthCanvas, heightCanvas) で描画領域を指定している．関数 draw() が呼び出されると変数 n を 1 フレーム毎に 1 ずつ増やしながら、その都度フレームを描画し続ける．いじってみるとだんだん理解が深まると思う．本質的な部分は帰納関数 iterateTriangleS() である．

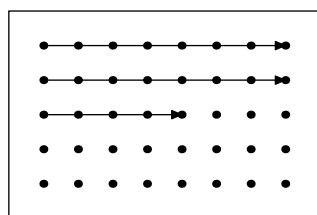


図 5.

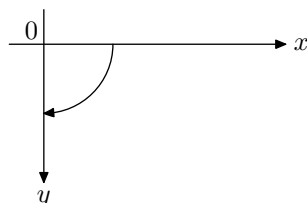


図 6.

p5.js 特有の関数を説明したい：

- `translate(a, b)`; (a, b) だけ平行移動し, その点を $(0, 0)$ として描画
- `rotate(theta)`; 角度 θ だけ直交座標系を回転し, それを新たな直交座標系として描画
- `triangle(a, b, a', b', a'', b'')`; (a, b) , (a', b') , (a'', b'') を頂点とする三角形を描画
- `background([a, b, c])`; 画面を RGB $[a, b, c]$ で埋める. ただし a, b, c は 0 から $255 = 2^8 - 1$ の間の整数.
- `fill([a, b, c])`; 次の図形を RGB $[a, b, c]$ で埋める.
- `push()`, `pop()` はその間にええられた設定を元に戻すためのスタックに書き込むための関数

関数 `iterateTriangleS(num)` がこのプログラムの本体である. `push()` でそれまで呼び込まれた設定を保存し, `pop()` で元の設定に戻っている. y 軸が逆だと分かりづらいので, y 軸を通常のようにとり, 回転方向も反時計回りだと思って正しくプログラムすると (図 7), 出力は y 軸が逆になって描画される. 三角形の重心が原点にあるとする. $\text{num} = 0$ のとき `fill(frontColor)` で次の図を `frontColor` で満たすことを, `noStroke()` で輪郭を描かないことを指示し,

```
triangle(-baseSegment3 / 2, -radius3 / 2,
        baseSegment3 / 2, -radius3 / 2, 0, radius3 );
```

で, 辺の長さが `baseSegment3` でその底辺が x 軸に平行な正三角形を書いている. このとき, `radius3` は正三角形の外接円の半径である.

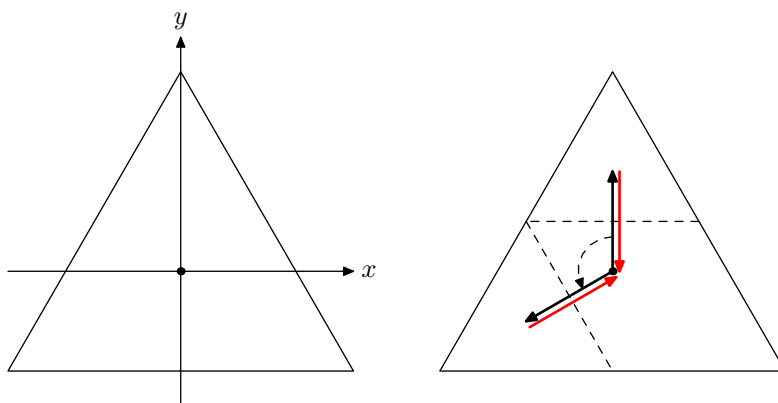


図 7.

num = 1 のときは scale $1/2$ にして $(0, 0)$ から $(0, \text{radius}3)$ 進むので実際は $(0, 0)$ から $(0, \text{radius}3/2)$ 進んで三角形を描く．その後 $(0, -\text{radius}3)$ 進み $(0, 0)$ に戻り $2\pi/3$ 回転する．これを三回繰り返して、3つの三角形を描く．

num > 1 のときはこれを繰り返すことで Sierpinski gasket を書くことが出来る．

レポート. *Sierpinski Carpet*(図 8) を描くようにプログラムを変更せよ．

レポート. *Koch flake*(図 9) を描くようにプログラムを変更せよ．

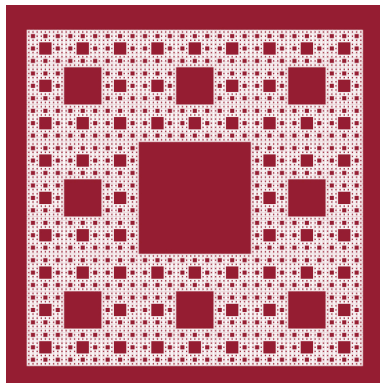


図 8. S. Carpet

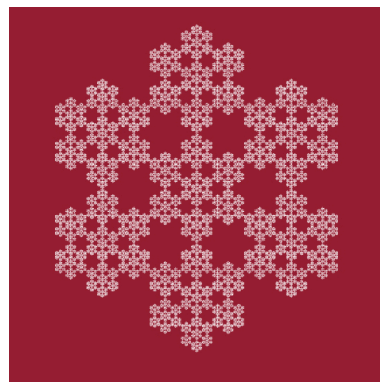


図 9. Koch flake

3. P5.JS の 3D-アプリ

3.1. ソース. 3次元の描画を行うことも出来る．createCanvas を createCanvas(widthCanvas, heightCanvas, WEBGL); に替えれば良い．ただし，WEBGL とはブラウザ上で CG を描画するための API である．以下がソースファイル sketch1.js の本質的な内容である．

```
/* Cubes landing before the Great Pyramids abbreviated version
 * 2026/06/17, by snow00two,
 * \href{https://creativecommons.org/licenses/by-nc-nd/4.0/}{\ccbyncsa}
 */
const WIDTH_CANVAS = 1080 ; // 720x3/2
const HEIGHT_CANVAS = 607.5 ; // 405x3/2
const BACK_COLOR = [150, 200, 250] ;//[200, 220, 200]

let octahedron ;
function preload(){
  octahedron = loadModel('octahedron.stl');
  //https://www.printables.com/model/411768-octahedron-freecad/files
```



```

}

function setup(){//canvas : window of the projection coordinates
  createCanvas(WIDTH_CANVAS, HEIGHT_CANVAS, WEBGL);
  background(BACK_COLOR);
}

let i = 0;//basic parameter for frames

function draw(){
  let I = 9425; //change the priod condition to  $3 \times 2\pi / 0.002 = 9424.7\dots$ 

  if (i<I) {
  } else {
    i = 0;// reset basic parameter
  }

  // define the camera coordinates from a camera on the helicopter
  // of the following coordinates in the world coordinates :
  let cameraX = cos(0.5 + i * 0.002)*250 ;
  let cameraY = sin(0.5 + i * 0.002)*250;
  let cameraZ = 200+sin(0.5 + i * 0.002)*30;
  camera(cameraX,cameraY, cameraZ, 100, 0, 100, 0, 0, -1);
  // the output images are in the projection coordinates.

  //The world coordinates :
  background(BACK_COLOR);
  directionalLight(250, 200, 200, 0, -1, -1); //light from the sun
  fill(250, 200, 200);
  plane(10000,10000);

  stroke(0);
  strokeWeight(2);
  fill(150,150, 150);
  pyramid(3);

  translate(-250,0,0);
  pyramid(3);

  translate(-250,0,0);
  pyramid(1.5);

  ambientMaterial(125);
  ambientLight(100,100,100);
  shininess(100.0);

```

```

    translate(700, 200, max(500 - i * 0.3, 67) );
    sCube(5);

    let M = 25;
    for(let m = 0 ; m < M ; m++){
        translate(140,140, max(650 + 150*m - i * 0.3, 0));
        sCube(5);
    }
    i++;
}

// The modeling coordinates :
function pyramid(scaleN){
    push();
    rotateZ(QUARTER_PI);
    scale(scaleN);
    model(octahedron);
    pop();
}

function sCube(scaleN){
    push();
    rotateZ(QUARTER_PI);
    scale(scaleN);
    box(25);
    pop();
}

```

3.2. p5.js の 3D モードの説明とソースファイルの解説. 関数 `preload()` でディレクトリにある図形 `octahedron.stl` を読み込み, 関数 `setup()` の中でウィンドウ等を初期化する. 関数 `draw()` が呼び出されると変数 `i` を 1 フレーム毎に 1 ずつ増やししながら, その都度フレームを描画し続ける. いじってみるとだんだん理解が深まると思う.

p5.js 特有の組込関数の説明:

- `translate(a, b, c);` (a, b, c) だけ平行移動し, その点を $(0, 0, 0)$ として描画
- `rotateZ(theta);` z 軸を中心に角度 `theta` だけ直交座標系を回転し, それを新たな直交座標系として描画. `rotateX`, `rotateY` も同様.
- `scale(a)` 座標系を a 倍する
- `strokeWeight(a)` 太さ a で描く
- `background(a,b,c);` 背後の画面を RGB $[a,b,c]$ で埋める. ただし a, b, c は 0 から $255 = 2^8 - 1$ の間の整数.

- `directionalLight(a, b, c, d,e,f)` RGB[a, b, c] の光で向き (d,e,f) $-1 \leq d, e, f \leq 1$ で光を照らす
- `ambientLight(a, b, c)` RGB[a, b, c] の光で背景となる光を照らす
- `ambientMaterial(a, b, c)` 次の図形の壁の色を RGB [a,b,c] で設定
- `fill(a,b,c)` 次の図形を RGB [a,b,c] で埋める.
- `plane(a,b)`, `box(c)` などは 3 次元の組込立体図形
- `camera(a, b, c, a', b', c')` カメラの位置 (a,b,c) 対象の中心 (a',b',c') に向けてウィンドウを描画
- `push()`, `pop()` はその間に変えられた設定を元に戻すためのスタックに書き込むための関数
- `model(modelName)` `modelName` を描く

レポート. *GitHub* のアカウントがあれば *GitHub* の *Pages* という機能を使うと自分のホームページが作れます. 私のポートフォリオもそのようにして作ったものです. やり方を調べて自分のホームページを作って, 自分の作った作品を公開してください.

REFERENCES

- [1] 松田晃一, p5.js プログラミングガイド 改訂版, 2021, カットシステム
- [2] 山田祥寛, 改訂 3 版 JavaScript 本格入門, 2023, 技術評論社